SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Smart University Laboratory Equipment Monitoring and Allocation System Using C

Course: C Programming | Course Code: CSA0211

Faculty: Dr S.K. Saravanan

Academic Year: 2026 – 2027

Submitted By

B Reshini Priya – 192320035

Digvijay A – 192320045

S Kavya – 192312393

1. Problem Statement

University laboratories contain many instruments such as digital multimeters, oscilloscopes, function generators, development boards, sensors, and power supplies. When equipment is managed manually, it is difficult to identify available items, track who has borrowed an item, prevent duplicate allocation, and identify equipment that needs maintenance. Manual records may also become outdated or contain errors.

The problem addressed by this project is to develop a simple C-based system that monitors laboratory equipment availability and supports controlled allocation and return of equipment. The system should also provide a mechanism to mark equipment as being under maintenance.

2. Objective

- To maintain a digital record of laboratory equipment.
- To display equipment ID, name, quantity, availability, and status.
- To allocate available equipment to students or staff.
- To process returned equipment and update availability.
- To mark equipment as unavailable when it requires maintenance.
- To reduce manual errors and duplicate allocation.
- To demonstrate the practical application of C programming concepts.
- To provide a base for future RFID, IoT, sensor, and database integration.

3. Requirements and Environment Used

3.1 Hardware Requirements

- Computer or laptop.
- Minimum 2 GB RAM recommended.
- Keyboard and monitor for console operation.
- Optional microcontroller, LCD, RFID reader, or sensors for future embedded implementation.

3.2 Software Requirements

- C programming language.
- GCC / Code::Blocks / Dev-C++ / Visual Studio Code with C compiler.
- Windows or Linux operating system.
- Text editor or C development IDE.

3.3 Programming Concepts Used

- Structures
- Arrays
- Functions
- Loops
- Conditional statements
- String handling
- Menu-driven programming

4. Design / Proposed Solution

The proposed system is a menu-driven laboratory equipment management application. Each equipment record contains an identification number, equipment name, total quantity, available quantity, maintenance status, and allocation information.

The operator can select an operation from the main menu. During allocation, the system checks whether the requested equipment exists, is not under maintenance, and has at least one available unit. If all conditions are satisfied, the available quantity is reduced. During return, the available quantity is increased up to the total quantity. Equipment can also be marked for maintenance.

Data Field	Example	Purpose
Equipment ID	101	Unique identification
Name	Digital Multimeter	Equipment name
Total	5	Total number of units
Available	4	Units currently available
Maintenance	No	Indicates maintenance condition
Allocated To	Student A	Current user

5. Algorithm / Pseudocode / Flowchart

5.1 Algorithm

1. Start.

2. Initialize the laboratory equipment records.

3. Display the main menu.

4. Read the user's choice.

5. If choice = 1, display all equipment details.

6. If choice = 2, request equipment ID and user name.

7. Check equipment availability and maintenance status.

8. If available, allocate the equipment and decrease available quantity.

9. If choice = 3, request equipment ID and process the return.

10. If choice = 4, mark the selected equipment for maintenance.

11. If choice = 5, terminate the program.

12. Otherwise, display an invalid-choice message.

13. Repeat until Exit is selected.

14. Stop.

5.2 Pseudocode

```
START
Initialize equipment records

REPEAT
    Display menu
    Read choice

    IF choice = 1
        Display equipment records

    ELSE IF choice = 2
        Read equipment ID and user name
        IF equipment exists AND available > 0 AND not in maintenance
            Decrease available quantity
            Store user name
            Display allocation success
        ELSE
            Display allocation failure
        ENDIF

    ELSE IF choice = 3
        Read equipment ID
        IF equipment exists AND available < total
            Increase available quantity
            Clear allocation user
```

```
            Display return success
        ELSE
            Display error
        ENDIF

    ELSE IF choice = 4
        Read equipment ID
        Mark equipment for maintenance

    ELSE IF choice = 5
        Exit program

    ELSE
        Display invalid choice
    ENDIF
UNTIL choice = 5

STOP
```

5.3 Flowchart

START → Initialize Equipment → Display Menu → Select Operation → Check Equipment/Availability → Perform Allocation / Return / Maintenance / Display → Show Result → Return to Menu → EXIT → STOP

6. Implementation / Source Code

```
#include <stdio.h>
#include <string.h>

#define MAX 10

struct Equipment {
    int id;
    char name[40];
    int total;
    int available;
    int maintenance;
    char allocatedTo[40];
};

struct Equipment e[MAX] = {
    {101, "Digital Multimeter", 5, 5, 0, ""},
    {102, "Oscilloscope", 3, 3, 0, ""},
    {103, "Function Generator", 2, 2, 0, ""},
    {104, "Arduino Board", 10, 10, 0, ""}
};

int n = 4;

void display(void)
{
    int i;
    printf("\nID\tEquipment\t\tTotal\tAvailable\tStatus\n");
```

```
    for (i = 0; i < n; i++) {
        printf("%d\t%-22s%d\t%d\t\t%s\n",
               e[i].id, e[i].name, e[i].total,
               e[i].available,
               e[i].maintenance ? "Maintenance" : "Available");
    }
}

void allocateEquipment(void)
{
    int id, i;
    char user[40];

    printf("\nEnter Equipment ID: ");
    scanf("%d", &id);

    for (i = 0; i < n; i++) {
        if (e[i].id == id) {

            if (e[i].maintenance) {
                printf("Equipment is under maintenance.\n");
                return;
            }

            if (e[i].available <= 0) {
                printf("Equipment is not available.\n");
                return;
            }

            printf("Enter user name: ");
            scanf(" %[^\n]", user);

            e[i].available--;
            strcpy(e[i].allocatedTo, user);

            printf("Equipment allocated successfully.\n");
            return;
        }
    }

    printf("Equipment ID not found.\n");
}

void returnEquipment(void)
{
    int id, i;

    printf("\nEnter Equipment ID: ");
    scanf("%d", &id);

    for (i = 0; i < n; i++) {
        if (e[i].id == id) {

            if (e[i].available < e[i].total)
                e[i].available++;

            e[i].allocatedTo[0] = '\0';

            printf("Equipment returned successfully.\n");
            return;
        }
    }

    printf("Equipment ID not found.\n");
}
```

```
void maintenance(void)
{
    int id, i;

    printf("\nEnter Equipment ID: ");
    scanf("%d", &id);

    for (i = 0; i < n; i++) {
        if (e[i].id == id) {
            e[i].maintenance = 1;
            printf("Equipment marked for maintenance.\n");
            return;
        }
    }

    printf("Equipment ID not found.\n");
}

int main(void)
{
    int choice;

    do {
        printf("\n===== SMART LAB EQUIPMENT SYSTEM =====\n");
        printf("1. Display Equipment\n");
        printf("2. Allocate Equipment\n");
        printf("3. Return Equipment\n");
        printf("4. Mark Maintenance\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                display();
                break;

            case 2:
                allocateEquipment();
                break;

            case 3:
                returnEquipment();
                break;

            case 4:
                maintenance();
                break;

            case 5:
                printf("Exiting...\n");
                break;

            default:
                printf("Invalid choice.\n");
        }

    } while (choice != 5);

    return 0;
}
```

7. Test Cases and Expected / Actual Results

Test Case	Input	Expected Result	Actual Result	Status
Display equipment	1	All records displayed	All records displayed	Pass
Allocate equipment	ID 101, Student A	Available count decreases	Available count decreased	Pass
Allocate unavailable item	No available units	Allocation rejected	Allocation rejected	Pass
Return equipment	ID 101	Available count increases	Available count increased	Pass
Maintenance update	ID 102	Status becomes Maintenance	Status updated	Pass
Invalid ID	999	Error displayed	Error displayed	Pass
Exit	5	Program terminates	Program terminated	Pass

Table 1 Testcases and results

8. Execution Screenshots / Output

The following sample console output represents the expected execution of the program. Actual screenshots can be inserted in this section after running the program in the selected C IDE.

```
===== SMART LAB EQUIPMENT SYSTEM =====
1. Display Equipment
2. Allocate Equipment
3. Return Equipment
4. Mark Maintenance
5. Exit
Enter choice: 1

ID      Equipment               Total   Available       Status
101     Digital Multimeter      5       5               Available
102     Oscilloscope            3       3               Available
103     Function Generator      2       2               Available
104     Arduino Board           10      10              Available

Enter choice: 2
Enter Equipment ID: 101
Enter user name: Student A
```

```
Equipment allocated successfully.

Enter choice: 1
ID      Equipment               Total   Available       Status
101     Digital Multimeter      5       4               Available

Enter choice: 3
Enter Equipment ID: 101
Equipment returned successfully.
```

Screenshot Placeholder 1: Main menu and equipment list

[Insert execution screenshot here]

Screenshot Placeholder 2: Successful equipment allocation

[Insert execution screenshot here]

Screenshot Placeholder 3: Equipment return / maintenance update

[Insert execution screenshot here]

9. Analysis and Discussion

The implementation demonstrates that a simple C program can provide basic laboratory resource management without requiring a database or complex software platform. Structures provide a convenient way to group equipment attributes, while functions separate the major operations and improve program readability.

The allocation operation checks availability before changing the record, which reduces the chance of allocating an unavailable item. The return operation restores availability, and the maintenance function prevents an item from being allocated after it has been marked for maintenance.

For a real university deployment, the program should be enhanced with persistent storage, authentication, multiple-user support, transaction history, and hardware or network interfaces. Sensors and RFID can be used to automate equipment identification and status monitoring.

10. Conclusion

The Smart University Laboratory Equipment Monitoring and Allocation System using C successfully provides a basic solution for recording, monitoring, allocating, returning, and maintaining laboratory equipment. The

project applies important C programming concepts such as structures, arrays, functions, loops, conditions, and string handling to a practical university laboratory problem.

The proposed system can serve as the software foundation for a more advanced smart laboratory solution using RFID, sensors, IoT communication, databases, and web or mobile interfaces.

11. Individual Contribution of Group Members

Group Member	Contribution	Signature
B Reshini Priya	Problem analysis	_____
Digvijay A	Coding	_____
S Kavya	testing / documentation	_____

Table 2 Team and contributions

Suggested distribution: Member 1 – problem definition and requirements; Member 2 – system design and algorithm; Member 3 – C implementation and testing; Member 4 – documentation, screenshots, and presentation. Modify the entries according to the actual work performed by each group member.

12. References

- Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language, Prentice Hall.
- C Standard Library documentation for input/output and string handling.
- Embedded C programming laboratory manuals and course notes.
- Microcontroller and embedded-system programming reference materials.

13. Appendices

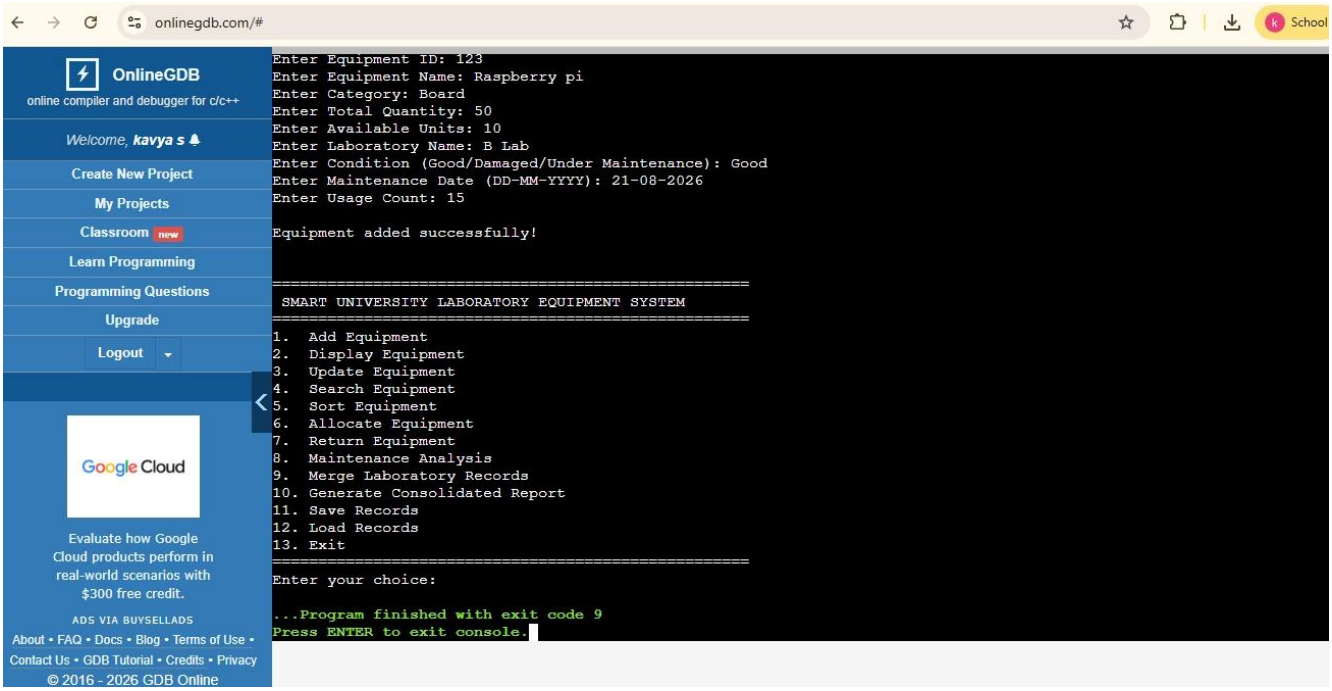

Figure 1 : Output for Add Equipment

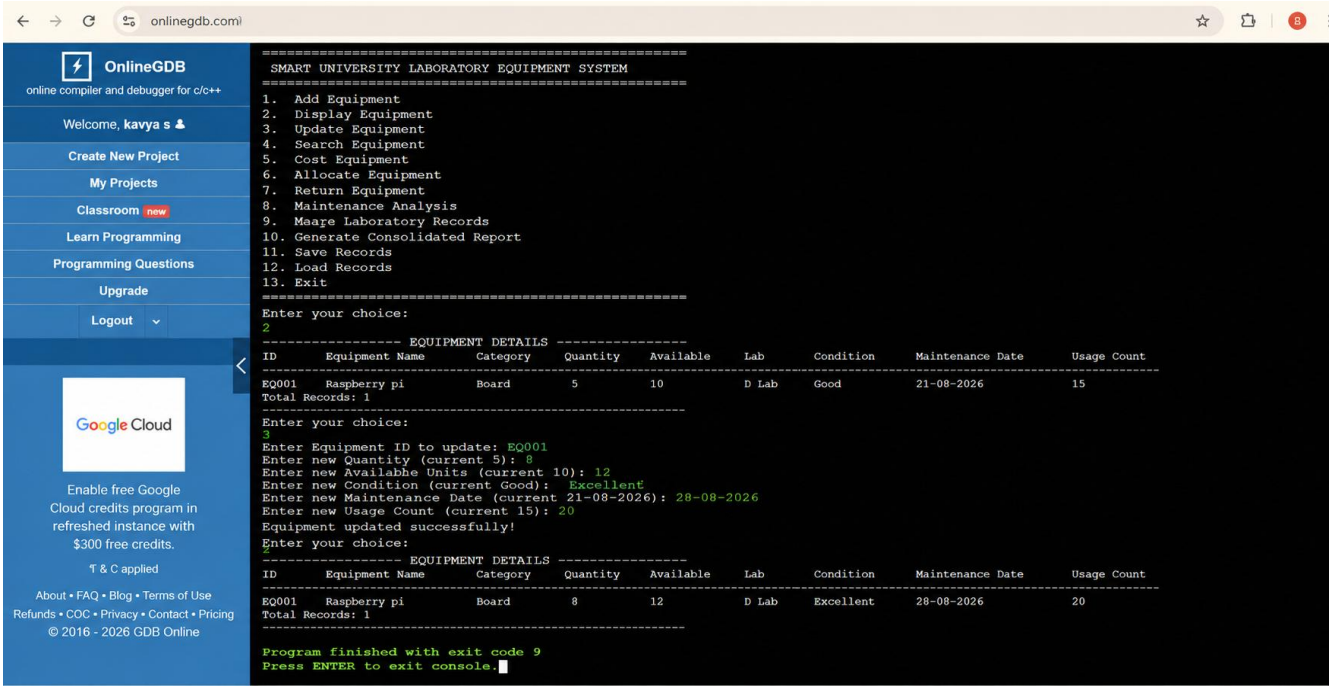

Figure 2 Update equipment output

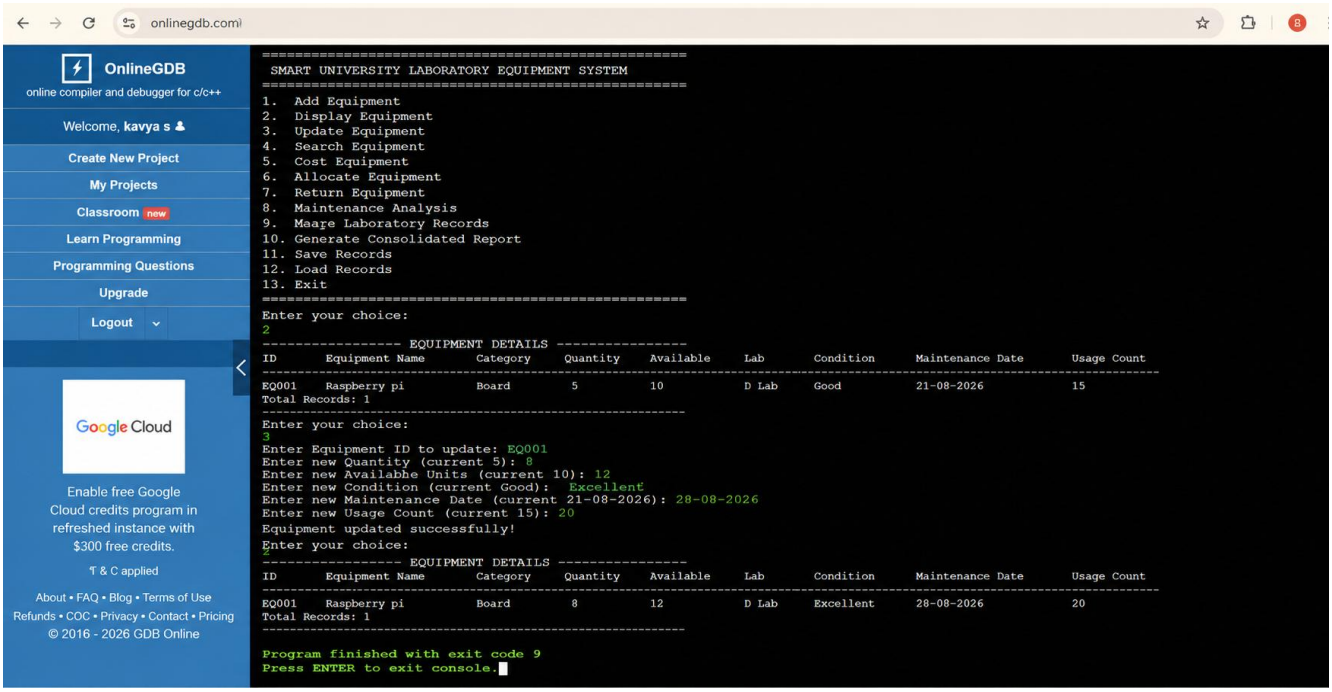

Figure 3 Delete equipment feature output

Figure 4 Team geotag